

Expectimax Search and Temporal-Difference Learning for the Game 2048

1st Emir Baručija

Faculty of Electrical Engineering, Sarajevo

Abstract—This project implements the game 2048 in C++ together with five playing agents: a random and a fixed-priority baseline, expectimax search with a hand-written evaluation, an n-tuple network value function trained by temporal-difference learning through self-play, and a combination in which the search evaluates its leaves with the learned values. The board is a 64-bit integer with four bits per cell, and moves are single table lookups, which is what makes both the deep search and n-tuple network training efficient. The learned agent reaches the 2048 tile in 95 percent of games and the 8192 tile in 44 percent, close to the results of the paper that introduced the method. A native GUI shows the agents playing and a console program trains the value function.

Index Terms—2048, expectimax, n-tuple network, temporal-difference learning, reinforcement learning

I. INTRODUCTION

2048 is played on a 4×4 grid. A move slides all tiles in one direction, equal tiles that collide merge into their sum, and after every move a new tile appears on a random empty cell, a 2 with probability 0.9 and a 4 with probability 0.1. The game is a Markov decision process with one player and a random environment: the player's move is deterministic, the spawn is not. This makes it a good target for expectimax search on one side and for reinforcement learning on the other, and the two approaches can be combined. The strongest published programs [1], [2] are built exactly that way, so I implemented both and then put them together.

II. IMPLEMENTATION

A. Game core

The board is one `uint64_t`: sixteen cells of four bits each, holding the rank of the tile (0 for empty, rank n for the tile 2^n). Sliding a row is a lookup in a precomputed table of 2^{16} entries that stores the resulting row and the points scored; a move in any other direction rotates or mirrors the board, slides left, and transforms back. The whole game state fits in a register, copying a position is free, and the board itself is the key of the transposition table. Both the search and the training loop depend on this: a move costs a few shifts and four lookups, nothing else.

B. Expectimax

The search tree alternates max nodes, where the agent chooses among the legal moves, and chance nodes, where every empty cell spawns a 2 and a 4 with their probabilities and the value is the weighted average [3]. Depth is adaptive: six plies when the board is open, and up to ten when two or

fewer cells are empty, where the chance nodes have almost no branches and every move decides the game. Iterative deepening runs the search at depth one, two, and so on up to the chosen depth, and puts the best move of the previous round first in the next one. A transposition table keyed by board, remaining depth and node type stores every evaluated position; the board is compared on a hit, since the key is a mix of the three and can collide. Branches whose cumulative probability falls below 10^{-4} are cut and evaluated directly. I first used 10^{-3} , which looked reasonable, but with it the search effectively stopped after three chance levels whatever depth was requested.

The evaluation is a weighted sum of four terms computed per row and per column: empty cells, available merges, monotonicity as a cost, and a clutter penalty, plus a large penalty for a lost position. The weights follow the widely used open-source expectimax player [4]. The important part is not the weights but the exponents: monotonicity is measured on rank^4 and clutter on $\text{rank}^{3.5}$, so breaking the order next to a large tile costs far more than next to a small one. An earlier version of mine used log-scaled terms with seven components, and it mismanaged exactly the large tiles that decide the endgame. All powers are read from a sixteen-entry table, so the leaf evaluation contains no calls to `pow` or `log`.

C. N-tuple network

The learned agent follows Szubert and Jaśkowski [1]. An n-tuple is a fixed set of board cells; its table has 16^n entries, indexed by packing the ranks of those cells together, which on the bitboard is again a few shifts. The value of a position is a plain sum of lookups,

$$V(s) = \sum_p \sum_{t \in D_4} W_p [\text{index}(p, t(s))], \quad (1)$$

where t runs over the eight symmetries of the square. All eight symmetric copies of a pattern share one table, which multiplies the training signal per board by eight and puts the symmetry of the game into the model instead of making it learn it. I use eight patterns of five cells: rows, columns and 2×2 blocks with one extension, placed towards the top-left corner, since the symmetries cover the rest.

The agent plays greedily one move deep, choosing the legal move that maximizes reward plus V of the afterstate, the board after sliding but before the random tile appears. Learning is TD(0) [5] on afterstates,

$$V(s'_t) \leftarrow V(s'_t) + \alpha [r_{t+1} + V(s'_{t+1}) - V(s'_t)], \quad (2)$$

and each correction is split evenly over the 64 weights the board touches. Learning on afterstates removes the spawn randomness from the target, which is why it converges much faster than learning the value of states. Action selection stays greedy the whole time; the random spawns supply enough exploration. The same class computes the action during training and during play, so the value function is never evaluated under a policy it has not seen.

The fifth agent keeps the expectimax search but evaluates its leaves with (1) instead of the hand-written terms. Nothing else changes; plain expectimax stays available as a clean baseline.

III. TRAINING AND RESULTS

The trainer is a console program that plays self-play games, prints the mean score and the tile rates every K games. Table I shows one run of 200,000 games with $\alpha = 0.1$.

TABLE I
SELF-PLAY TRAINING, LEARNING ON

Games	Mean score	2048	4096	8192
20,000	27,446	50%	10%	—
60,000	58,733	92%	63%	3%
100,000	67,354	95%	77%	7%
160,000	86,084	96%	83%	33%
200,000	93,131	96%	85%	43%

Measured afterwards on 300 games with learning switched off, the greedy agent scores 91,304 on average, with a best game of 173,236, and reaches 2048 in 95%, 4096 in 83% and 8192 in 44% of games. The 16384 tile appeared during training, in fewer than one game in two hundred. Szubert and Jaśkowski report mean scores around 100,000 for this configuration, and the curve was still rising at the end of the run, so more games would close the gap.

IV. APPLICATION

I built the GUI with the natID framework. A dropdown switches between human and AI mode; switching resets the board so a score is never carried over between the two. In human mode the arrow keys move the tiles. In AI mode the agent is picked from a second dropdown and driven by a timer at five moves per second, with Start, Stop, Continue and a single-step button for following one decision. The panel shows score, best tile, move count and separate best scores for human and AI, and for the search agents the depth, node count and time of the last move. An Evil Tiles toggle replaces the random spawn with the worst tile according to the evaluation, which is the quickest way to see how robust an agent is. The GUI loads the trained weights at startup and shows how many games produced them; the two learned agents are refused until the file exists.

REFERENCES

- [1] M. Szubert and W. Jaśkowski, “Temporal difference learning of N-tuple networks for the game 2048,” in Proc. IEEE Conf. Computational Intelligence and Games (CIG), Dortmund, Germany, 2014, pp. 1–8.

- [2] W. Jaśkowski, “Mastering 2048 with delayed temporal coherence learning, multistage weight promotion, redundant encoding, and carousel shaping,” IEEE Trans. Games, vol. 10, no. 1, pp. 3–14, Mar. 2018.
- [3] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2021, ch. 5.
- [4] R. Xiao, “2048-ai: an AI for the 2048 game using expectimax optimization,” GitHub repository, 2014. [Online]. Available: <https://github.com/nneonneo/2048-ai>
- [5] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018, ch. 6.